PROJECT 1

Muhammet Ali Ayvaz

210102002078

Libraries(numpy, matplotlib, random) were first written to use functions

In [25]:
```
import numpy as np                          # for linespace
from matplotlib import pyplot as plt        # for plot, xlabel, y label, grid, sh
import random                               # to produce random numbers only for
from tabulate import tabulate               # to print as a tablo
import numpy as np
import matplotlib.pyplot as plt
```

Then, it was created fx_function and d_fx functions to calculate f(x) and f'(x) generically and it is written.

$f_(x) = 0.25x^2 - 2.75x + 6.66$

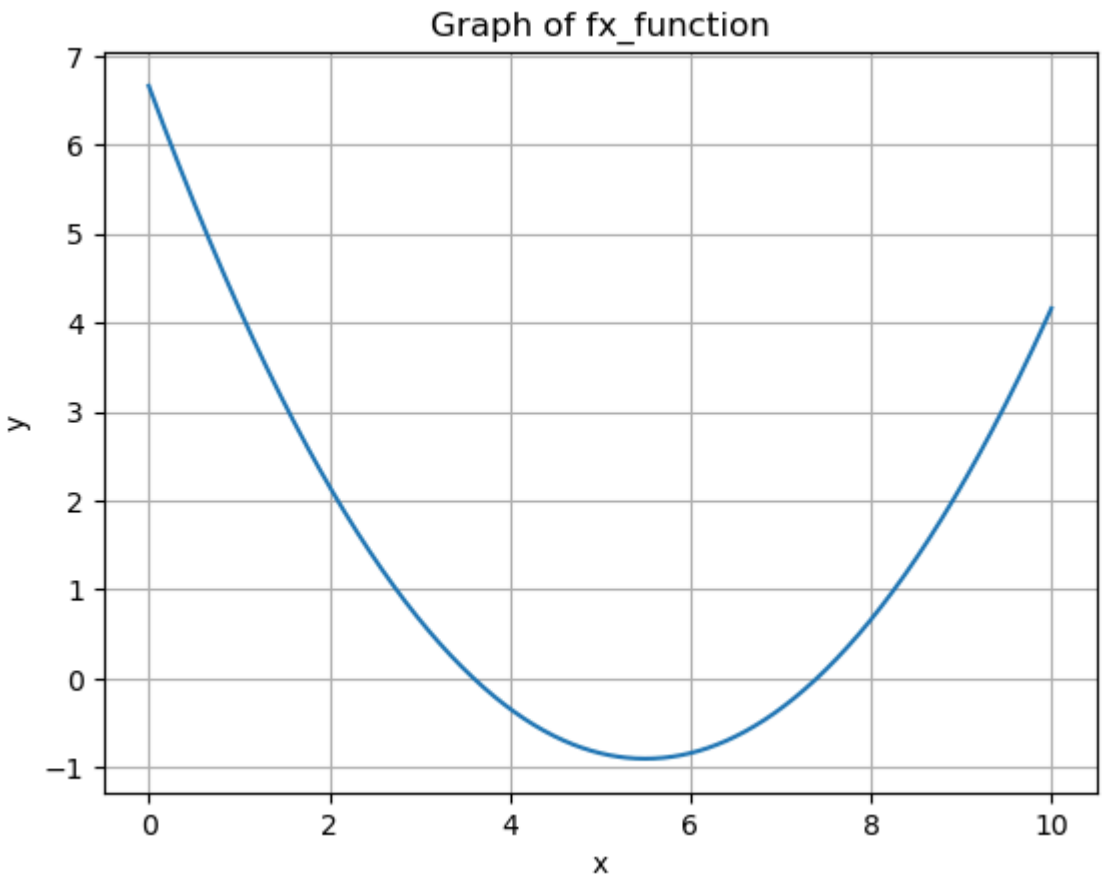
$f'_(x) = 0.50x - 2.75$

In [26]:
```
def fx_function(x): # that calculates y
    y = 0.25*(x**2)-(2.75)*x+6.66
    return y

def d_fx(x):  # that calculates derivative of y
    y = (0.5) * x - 2.75
    return y
```

1-)

A) Section a says that you must show to us f(x) function by using numpy and pyplot. Additioally, you must to guess the roots of f(x) funstions.

In [27]:
```
my_list = []

# 1-)

#section a according to project file.

def section_A(a, b):  # that shows fx function [a, b]
    x_values = np.linspace(a, b, 1000)  # x_values is a list which has 1000
    y_values = []  # empty list for y values

    for i in range(0, 1000):
        y_values.append(fx_function(x_values[i]))  # it is added y values wi

    plt.plot(x_values, y_values)  # x and y values are put with pyplot libra
    plt.xlabel("x")  # to write x
    plt.ylabel("y")  # to write y
    plt.grid(True)  # for grid
    plt.title("Graph of fx_function")  # optional: add a title
    plt.show()  # it shows

section_A(0, 10)  # (function call) it works to print totally
```



My root guess:

x1 = 3.8

x2 = 7.1

B) In b, the roots of f(x) were asked to be calculated. The function calculate the roots of f(x) analytically by using determinant and shows the real roots with print function.

In [28]:
```
#section b according to project file.
def section_B(a, b, c):  #that finds root with a,b,c parameters.
    dt = b * b - 4 * a * c
    if (dt > 0): #if determinant > 0 , there are two roots.
        x1 = (-b - (dt)**0.5) / (2 * a)
        x2 = (-b + (dt)**0.5) / (2 * a)
        print("for f(x)\nx1 = {}\nx2 = {}".format(x1, x2))
    elif (dt == 0): # if dt = 0, there is one
        x = (-b + (dt)**0.5) / (2 * a)
        print("there is only one, x = {}".format(x))
    elif (dt < 0): #if dt < 0, there is no any roots.
        print("no any roots")

section_B(0.25, -2.75, 6.66)
```

```
for f(x)
x1 = 3.6
x2 = 7.4
```

My root guess: x1 = 3.8, x2 = 7.1

Real roots: x1 = 3.6, x2 = 7.4

My guess and real roots are consistent.

C) Bisection Method

a,b parameters are (first x value, last x value)

In [29]:
```
def bisection(a, b, real_val): #that find roots with bisection method a,b =
    error = 0.000000000001 #error tolerance
    n = 1 # iteration count
    print("***********THAT'S BISECTION METHOD *************")
    while (1): # always true
        new_root = (a + b) / 2 # that finds new root by calculating (a + b)
        f_a = fx_function(a) # it is f(a)
        f_b = fx_function(b) # it is f(b)
        f_newroot = fx_function(new_root) #it is f(a + b / 2)
        if (f_newroot * f_a < 0): # if f(a) x f(a+b / 2)  < 0  , it is chang
            b = new_root
        else: #if not changed a
            a = new_root
        if ((b-a) / (2**n) < error): #if the expression < error , then it is
            print("it is founded...\nthe x{} : {}".format(n, new_root))
            print("iteration number : {} (with error : {})".format(n, error)
            my_list.append(["BISECTION", new_root, real_val])
            break
        else:# if not , the loop is continue
            print("{}. iteration \n".format(n))
            print("[  X{} = {}  ]\n".format(n, new_root))
            n += 1 # iteration number changed
            continue

bisection(2, 4, 3.6)
```

```
***********THAT'S BISECTION METHOD *************
1. iteration

[  X1 = 3.0  ]

2. iteration

[  X2 = 3.5  ]

3. iteration

[  X3 = 3.75  ]

4. iteration

[  X4 = 3.625  ]

5. iteration

[  X5 = 3.5625  ]

6. iteration

[  X6 = 3.59375  ]

7. iteration

[  X7 = 3.609375  ]

8. iteration

[  X8 = 3.6015625  ]

9. iteration

[  X9 = 3.59765625  ]

10. iteration

[  X10 = 3.599609375  ]

11. iteration

[  X11 = 3.6005859375  ]

12. iteration

[  X12 = 3.60009765625  ]

13. iteration

[  X13 = 3.599853515625  ]

14. iteration

[  X14 = 3.5999755859375  ]

15. iteration

[  X15 = 3.60003662109375  ]
```

```
16. iteration

[  X16 = 3.600006103515625  ]

17. iteration

[  X17 = 3.5999908447265625  ]

18. iteration

[  X18 = 3.5999984741210938  ]

19. iteration

[  X19 = 3.6000022888183594  ]

20. iteration

[  X20 = 3.6000003814697266  ]

it is founded...
the x21 : 3.59999942779541
iteration number : 21 (with error : 1e-12)
```

In [30]:
```
bisection(6, 8, 7.4) #function call to print
```

```
***********THAT'S BISECTION METHOD *************
1. iteration

[  X1 = 7.0  ]

2. iteration

[  X2 = 7.5  ]

3. iteration

[  X3 = 7.25  ]

4. iteration

[  X4 = 7.375  ]

5. iteration

[  X5 = 7.4375  ]

6. iteration

[  X6 = 7.40625  ]

7. iteration

[  X7 = 7.390625  ]

8. iteration

[  X8 = 7.3984375  ]

9. iteration

[  X9 = 7.40234375  ]

10. iteration

[  X10 = 7.400390625  ]

11. iteration

[  X11 = 7.3994140625  ]

12. iteration

[  X12 = 7.39990234375  ]

13. iteration

[  X13 = 7.400146484375  ]

14. iteration

[  X14 = 7.4000244140625  ]

15. iteration

[  X15 = 7.39996337890625  ]
```

```
16. iteration

[  X16 = 7.399993896484375  ]

17. iteration

[  X17 = 7.4000091552734375  ]

18. iteration

[  X18 = 7.400001525878906  ]

19. iteration

[  X19 = 7.399997711181641  ]

20. iteration

[  X20 = 7.399999618530273  ]

it is founded...
the x21 : 7.40000057220459
iteration number : 21 (with error : 1e-12)
```

That function takes one parameter as xn0(according to Newton Raphson methot formula) and starts infinite loop up to real root. For [2,4] Newton Raphson for x1: output:

In [31]:
```
def newton_method(xn0,real_val): # that find root of f(x) according to newto
    def formula(xn): ## that calculates x1 with f(x) and derivative of f(x)
        xn1 = xn - fx_function(xn) / d_fx(xn)
        return xn1

    error = 0.000000000001 # error tolerance
    n = 1 #iteration count
    print("***********THAT'S NEWTON RAPHSON METHOD*************")
    xn = xn0                          #first values
    xn1 = formula(xn) # calculates x1
    while (1): #infinite loop
        if (abs(xn1 - xn) < error): # it is calculated absolute(xn1 - xn2)
            print("\nit is founded.... with error tolerance : {}\n".format(e
            print("iteration number : {}".format(n))
            print("x{} = {}".format(n, xn1))
            my_list.append(["NEWTON RAPHSON", xn1, real_val])
            break  #if yes stops....
        else: # if no, continues
            print("{}. iteration".format(n))
            print("[  x{} = {}   ]".format(n, xn1))
            print("\n")
            n += 1 #iteration number increase + 1
            temp = xn1                       #### swapping old values to ne
            xn = xn1
            xn1 = formula(temp)



newton_method(2, 3.6) #function call to print
```

```
***********THAT'S NEWTON RAPHSON METHOD*************
1. iteration
[  x1 = 3.2342857142857144   ]


2. iteration
[  x2 = 3.570484597369843   ]


3. iteration
[  x3 = 3.5997742544601525   ]


4. iteration
[  x4 = 3.599999986590791   ]


5. iteration
[  x5 = 3.6   ]



it is founded.... with error tolerance : 1e-12

iteration number : 6
x6 = 3.6
```

In [32]:
```
newton_method(6, 7.4)
```

```
***********THAT'S NEWTON RAPHSON METHOD*************
1. iteration
[  x1 = 9.36   ]


2. iteration
[  x2 = 7.897616580310881   ]


3. iteration
[  x3 = 7.451639253547411   ]


4. iteration
[  x4 = 7.400683172492583   ]


5. iteration
[  x5 = 7.400000122778131   ]


6. iteration
[  x6 = 7.400000000000007   ]



it is founded.... with error tolerance : 1e-12

iteration number : 7
x7 = 7.400000000000001
```

Secant Methot Functions takes two parameters as x0 and x1 (firstvalues according to formula of secant method) and calculates candidate root unlike Newtob Raphson method. x1 means x(-1) = 1 and x(0) = 2

In [33]:
```
def secant_method(x1, x0, real_val): #that function takes x0, x1 from user a
    def formula(x0, x1): #like newton raphson method but there are f(x) proc
        x2 = x1 - (fx_function(x1) * (x1 - x0)) / (fx_function(x1) - fx_func
        return x2

    error = 0.000000000001 # error tolerance
    n = 0 #iteration count
    print("***********THAT'S SECANT METHOD*************")
    x0, x1 = x0, x1 #swapping for other loop.
    x2 = formula(x0, x1) #new root
    while (1): #infinite loop
        if (abs(x2 - x1) < error): #if yes, it stops
            print("\nIt is found with error tolerance: {}\n".format(error))
            print("Iteration number: {}".format(n + 1))
            print("x{} = {}".format(n + 1, x2))
            my_list.append(["SECANT", x2, real_val])
            break
        else: #if not , continues for other candidate values
            n += 1
            print("{}. iteration".format(n))
            print("[x{} = {}]]".format(n, x2))
            x0, x1 = x1, x2 #swapping for other loop
            x2 = formula(x0, x1) #new candidate root

secant_method(2, 3, 3.6) #function call x(-1) = 1, x(0) = 2
```

```
***********THAT'S SECANT METHOD*************
1. iteration
[x1 = 3.4400000000000004]]
2. iteration
[x2 = 3.5539568345323747]]
3. iteration
[x3 = 3.5981610516485887]]
4. iteration
[x4 = 3.599977995426906]]
5. iteration
[x5 = 3.5999999893564563]]
6. iteration
[x6 = 3.599999999999939]]

It is found with error tolerance: 1e-12

Iteration number: 7
x7 = 3.6000000000000005
```

In [34]:
```
secant_method(6, 7, 7.4) #function call x(-1) = 6, x(0) = 7
```

```
***********THAT'S SECANT METHOD*************
1. iteration
[x1 = 7.68]]
2. iteration
[x2 = 7.2537313432835795]]
3. iteration
[x3 = 7.389588708453482]]
4. iteration
[x4 = 7.4004179829407155]]
5. iteration
[x5 = 7.399998851785073]]
6. iteration
[x6 = 7.399999999873713]]
7. iteration
[x7 = 7.399999999999999]]

It is found with error tolerance: 1e-12

Iteration number: 8
x8 = 7.399999999999999
```

Fixed Point İteraiton Method:

Firstly, it was written two functions (if it doesn't work: x = g(x) or x = k(x) according to fixed point iteration method). That wto functions was written global namespace, it was done to be generic code.

In [35]:
```
def g1(x):
    y = (0.25*x**2 + 6.66) / 2.75
    return y

def g2(x):
    y = (2.75*x - 6.66)**0.5
    return y

#it will be used only one....

def fixed_point_i_method(a, b, function_no, real_val): # a and b mean first
    print("********THAT'S FIXED POINT METHOD************************")
    error = 0.000000000001 #error tolerance
    if (function_no == 1): #if function_no 1 , it will be chosen g1 . vice v
        g = g1
    else:
        g = g2

    def condition_test(xi, xi1): # it was written to find any suitable value
            if (0 < abs((g(xi) - g(xi1)) / (xi - xi1)) < 1 ):  # abs is absc
                return xi1
            else:
                return None
    xi = random.uniform(a, b) # it is chosen random any float value between
    xi1 = g(xi) # either g1 or g2.
    while (1): #infinite loop
        if (condition_test(xi, xi1)): # if  0 < .. < 1 , stops if not, conti
            break
        xi = random.uniform(a, b)
        xi1 = g(xi)
    # chosed real value.....
    n = 1 #iteration counter
    print("it is chosed random (x0) : {} for [{}, {}] (because it is suitabl
    while (1): #infinite loop
        if (abs(xi1 - xi) < error): #if expression suits < error
            print("\nIt is found with error tolerance: {}\n".format(error))
            print("x{} = {}".format(n, xi1))
            print("the iteration number : {}".format(n))
            my_list.append(["FIXED POINT", xi1, real_val])
            break #it breaks
        else:
            print("{}. iteration".format(n))
            print("[ x{} = {}]  ]".format(n, xi1))
            xi = xi1
            xi1= g(xi1)
            n +=1 #adds +1 iteration counter
#section d

fixed_point_i_method(2, 4, 1, 3.6) #function call [2, 4] first function
```

```
50. iteration
[ x50 = 3.5999999999505023]  ]
51. iteration
[ x51 = 3.599999999967601]  ]
52. iteration
[ x52 = 3.5999999999787935]  ]
53. iteration
[ x53 = 3.599999999986119]  ]
54. iteration
[ x54 = 3.599999999990914]  ]
55. iteration
[ x55 = 3.599999999994053]  ]
56. iteration
[ x56 = 3.599999999996107]  ]
57. iteration
[ x57 = 3.599999999997452]  ]

It is found with error tolerance: 1e-12

x58 = 3.599999999998332
```

In [36]:
```
fixed_point_i_method(6, 8, 2, 7.4) #function call [6, 8] second function
```

```
********THAT'S FIXED POINT METHOD************************
it is chosed random (x0) : 7.549954575185726 for [6, 8] (because it is sui
table according to 0 < ....  < 1 formula)
1. iteration
[ x1 = 3.755312913960799]  ]
2. iteration
[ x2 = 1.9149701077019967]  ]
3. iteration
[ x3 = (7.229131142584575e-17+1.180606710051874j)]  ]
4. iteration
[ x4 = (0.6120515771270217+2.6522833809877246j)]  ]
5. iteration
[ x5 = (1.3880039203020273+2.627434689135866j)]  ]
6. iteration
[ x6 = (1.5687019660206292+2.3030013194452175j)]  ]
7. iteration
[ x7 = (1.4845461948119079+2.133060476867401j)]  ]
8. iteration
[ x8 = (1.3837850584838662+2.1195185897628854j)]  ]
9. iteration
[ x9 = (1.348252067889982+2.161567655138044j)]  ]
10. iteration
[ x10 = (1.3573470653632664+2.1896798554018853j)]  ]
11. iteration
[ x11 = (1.3726157155211975+2.1934834106386094j)]  ]
12. iteration
[ x12 = (1.378644249919598+2.1876852493338887j)]  ]
13. iteration
[ x13 = (1.3777455336695263+2.183325689920639j)]  ]
14. iteration
[ x14 = (1.3755254296227104+2.1824916929847737j)]  ]
15. iteration
[ x15 = (1.3745186640521954+2.183255969043799j)]  ]
16. iteration
[ x16 = (1.3745775305642958+2.183926981698037j)]  ]
17. iteration
[ x17 = (1.3748968094204779+2.184090892683452j)]  ]
18. iteration
[ x18 = (1.3750613381912034+2.1839934656225206j)]  ]
19. iteration
[ x19 = (1.3750641173971507+2.1838916289338615j)]  ]
20. iteration
[ x20 = (1.3750189910960766+2.183861466080829j)]  ]
21. iteration
[ x21 = (1.3749925807363135+2.1838732498855555j)]  ]
22. iteration
[ x22 = (1.3749903964902357+2.1838885029725104j)]  ]
23. iteration
[ x23 = (1.3749966537339309+2.1838938178014424j)]  ]
24. iteration
[ x24 = (1.3750008263609064+2.1838925053044314j)]  ]
25. iteration
[ x25 = (1.3750014190998112+2.183890251371163j)]  ]
26. iteration
[ x26 = (1.375000571116291+2.1838893442767757j)]  ]
27. iteration
[ x27 = (1.3749999214035984+2.1838894691101243j)]  ]
28. iteration
[ x28 = (1.374999793249316+2.1838897974888214j)]  ]
29. iteration
[ x29 = (1.3749999049279782+2.1838899484901546j)]  ]
```

```
30. iteration
[ x30 = (1.375000004714254+2.183889941002582j)]  ]
31. iteration
[ x31 = (1.3750000296653622+2.183889893885575j)]  ]
32. iteration
[ x32 = (1.3750000155043942+2.183889869260201j)]  ]
33. iteration
[ x33 = (1.375000000381369+2.183889868654479j)]  ]
34. iteration
[ x34 = (1.3749999958151733+2.1838898753011704j)]  ]
35. iteration
[ x35 = (1.3749999975157805+2.1838898792468155j)]  ]
36. iteration
[ x36 = (1.37499999977755+2.183889879600129j)]  ]
37. iteration
[ x37 = (1.375000000578917+2.1838898786806444j)]  ]
38. iteration
[ x38 = (1.3750000003918317+2.1838898780583045j)]  ]
39. iteration
[ x39 = (1.375000000058123+2.183889877965989j)]  ]
40. iteration
[ x40 = (1.3749999999217677+2.1838898780902443j)]  ]
41. iteration
[ x41 = (1.3749999999390832+2.183889878186997j)]  ]
42. iteration
[ x42 = (1.3749999999876223+2.183889878206656j)]  ]
43. iteration
[ x43 = (1.3750000000102656+2.1838898781903517j)]  ]
44. iteration
[ x44 = (1.375000000009342+2.183889878175514j)]  ]
45. iteration
[ x45 = (1.37500000000239+2.183889878171718j)]  ]
46. iteration
[ x46 = (1.3749999999987046+2.1838898781737752j)]  ]
47. iteration
[ x47 = (1.374999999998586+2.1838898781760205j)]  ]
48. iteration
[ x48 = (1.3749999999995648+2.1838898781767115j)]  ]

It is found with error tolerance: 1e-12

x49 = (1.3750000000001543+2.1838898781764664j)
the iteration number : 49
```

Now print all conclusion

In [37]:
```
def print_all(): # prints all
    table = [["METHOD", "REAL ROOT", "ESTIMATED ROOT", "RELATIVE ERROR"]]
    for i in range(0, len(my_list)):
        table.append([my_list[i][0], my_list[i][1], my_list[i][2], (abs(my_l
    print(tabulate(table, headers = "firstrow", tablefmt = "fancy_grid", num



print_all()
```

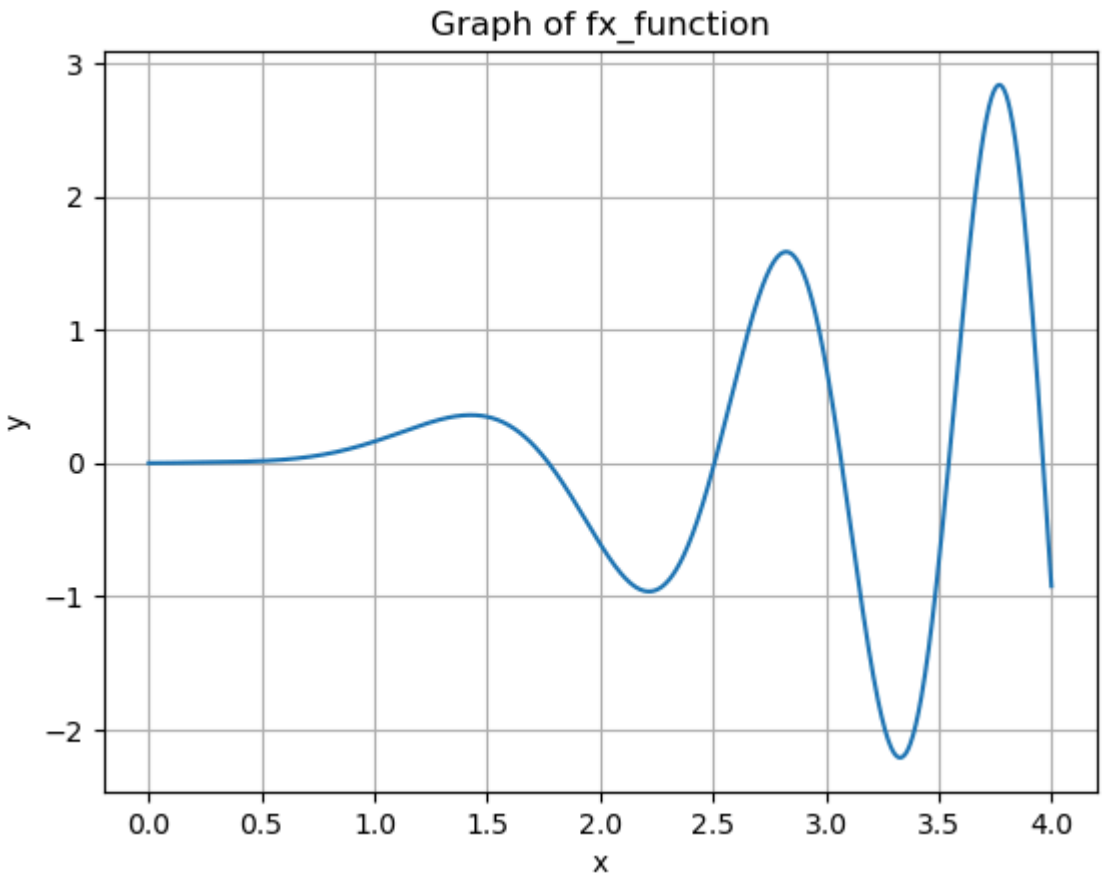
```
╒═══════════════╤═══════════════════════════════════════╤═══════
═══════════════╤═══════════════════════════════╕
│ METHOD        │ REAL ROOT                             │  ESTIMATED R
OOT   │ RELATIVE ERROR                    │
╞═══════════════╪═══════════════════════════════════════╪═══════
═══════════════╪═══════════════════════════════╡
│ BISECTION     │ 3.59999942779541                      │       3.6
│ 1.5894574468945904e-05                  │
├───────────────┼───────────────────────────────────────┼───────
───────────────┼───────────────────────────────┤
│ BISECTION     │ 7.40000057220459                      │       7.4
│ 7.732493854632349e-06                   │
├───────────────┼───────────────────────────────────────┼───────
───────────────┼───────────────────────────────┤
│ NEWTON RAPHSON │ 3.6                                   │       3.6
│ 0.0                                     │
├───────────────┼───────────────────────────────────────┼───────
───────────────┼───────────────────────────────┤
│ NEWTON RAPHSON │ 7.400000000000001                     │       7.4
│ 1.2002411077028718e-14                  │
├───────────────┼───────────────────────────────────────┼───────
───────────────┼───────────────────────────────┤
│ SECANT        │ 3.6000000000000005                    │       3.6
│ 1.233581138472396e-14                   │
├───────────────┼───────────────────────────────────────┼───────
───────────────┼───────────────────────────────┤
│ SECANT        │ 7.399999999999999                     │       7.4
│ 2.4004822154057446e-14                  │
├───────────────┼───────────────────────────────────────┼───────
───────────────┼───────────────────────────────┤
│ FIXED POINT   │ 3.599999999998332                     │       3.6
│ 4.6333307561044665e-11                  │
├───────────────┼───────────────────────────────────────┼───────
───────────────┼───────────────────────────────┤
│ FIXED POINT   │ (1.3750000000001543+2.1838898781764664j) │       7.4
│ (132.3094366892631-210.14490150754855j) │
╘═══════════════╪═══════════════════════════════════════╪═══════
═══════════════╪═══════════════════════════════╛
```

2. Electronics Engineering Application Scenario:

In [38]:
```
# 2-)

def fx_function(x): # that calculates y
    y = 0.2 * x * x * (np.cos(2.5 * x) * np.exp(-3.2 * x) + np.sin(x * x))
    return y

def d_fx(x):  # that calculates derivative of y
    a = 0.4 * x * (np.cos(2.5 * x) * np.exp(-3.2 * x) + np.sin(x * x))
    b = 0.2 * x * x * (2 * x * np.cos(x * x) + np.exp(-3.2 * x) * (-2.5 * np
    y = a + b
    return y

#def g1(x): # x = g(x) function according to fixed point iteration method  #
   # y = abs(np.arcsin(-np.cos(2.5 * x) * np.exp(-3.2 * x)))**0.5
    #return y
```

In [39]:
```
my_list = []
```

In [40]:
```
section_A(0, 4)#function call for showing new function
```



In [41]:
```
bisection(1, 2, 1.77)
```

```
***********THAT'S BISECTION METHOD *************
1. iteration

[  X1 = 1.5  ]

2. iteration

[  X2 = 1.75  ]

3. iteration

[  X3 = 1.875  ]

4. iteration

[  X4 = 1.8125  ]

5. iteration

[  X5 = 1.78125  ]

6. iteration

[  X6 = 1.765625  ]

7. iteration

[  X7 = 1.7734375  ]

8. iteration

[  X8 = 1.76953125  ]

9. iteration

[  X9 = 1.771484375  ]

10. iteration

[  X10 = 1.7724609375  ]

11. iteration

[  X11 = 1.77197265625  ]

12. iteration

[  X12 = 1.772216796875  ]

13. iteration

[  X13 = 1.7720947265625  ]

14. iteration

[  X14 = 1.77215576171875  ]

15. iteration

[  X15 = 1.772186279296875  ]
```

```
16. iteration

[  X16 = 1.7721710205078125  ]

17. iteration

[  X17 = 1.7721786499023438  ]

18. iteration

[  X18 = 1.7721824645996094  ]

19. iteration

[  X19 = 1.7721843719482422  ]

it is founded...
the x20 : 1.7721834182739258
iteration number : 20 (with error : 1e-12)
```

In [42]:
```
bisection(2, 3, 2.5)
bisection(3, 3.5, 3.07)
bisection(3.5, 3.8, 3.54)
bisection(3.8, 4, 3.96)
```

```
15. iteration

[  X15 = 3.963323974609376  ]

16. iteration

[  X16 = 3.963327026367188  ]

17. iteration

[  X17 = 3.963325500488282  ]

18. iteration

[  X18 = 3.963326263427735  ]

it is founded...
the x19 : 3.9633266448974616
iteration number : 19 (with error : 1e-12)
```

In [43]:
```
newton_method(2, 2.5)
```

```
***********THAT'S NEWTON RAPHSON METHOD*************
1. iteration
[  x1 = 1.7755653115437517   ]


2. iteration
[  x2 = 1.7721973261545945   ]


3. iteration
[  x3 = 1.772183494116004   ]


4. iteration
[  x4 = 1.7721835031194266   ]


5. iteration
[  x5 = 1.7721835031133883   ]



it is founded.... with error tolerance : 1e-12

iteration number : 6
x6 = 1.7721835031133923
```

In [44]:
```
newton_method(1, 1.7)
newton_method(3, 3.07)
newton_method(3.5, 3.54)
newton_method(3.7, 3.96)
```

```
3. iteration
[  x3 = 3.54506695456679   ]


4. iteration
[  x4 = 3.544909134266124   ]


5. iteration
[  x5 = 3.544909114950235   ]



it is founded.... with error tolerance : 1e-12

iteration number : 6
x6 = 3.5449091149500127
```

In [45]:
```
secant_method(1.5, 2, 2.5)
```

```
***********THAT'S SECANT METHOD*************
1. iteration
[x1 = 1.6822668690442568]]
2. iteration
[x2 = 1.861828311116245]]
3. iteration
[x3 = 1.7608475090494586]]
4. iteration
[x4 = 1.7710018304553325]]
5. iteration
[x5 = 1.7722031325629135]]
6. iteration
[x6 = 1.772183470050922]]
7. iteration
[x7 = 1.77218350311247]]

It is found with error tolerance: 1e-12

Iteration number: 8
x8 = 1.7721835031133923
```

In [46]:
```
secant_method(0.5, 1, 1.7)
secant_method(2.8, 3, 3.07)
secant_method(3.3, 3.4, 3.54)
secant_method(3.6, 3.8, 3.96)
```

```
x11 = 3.069981694013833
***********THAT'S SECANT METHOD*************
1. iteration
[x1 = 3.4871888222253444]]
2. iteration
[x2 = 3.54261823590564]]
3. iteration
[x3 = 3.5450747876204827]]
4. iteration
[x4 = 3.544908840263008]]
5. iteration
[x5 = 3.544909114917986]]
6. iteration
[x6 = 3.5449091149500127]]

It is found with error tolerance: 1e-12

Iteration number: 7
x7 = 3.5449091149500127
```

In [47]:
```
print_all()

#fixed_point_i_method(1, 2, 1)
#fixed_point_i_method(2, 2.9, 1)
#fixed_point_i_method(3, 3.4, 1)
#fixed_point_i_method(3.5, 3.8, 1)
#fixed_point_i_method(3.8, 4., 1)
```

METHOD	REAL ROOT	ESTIMATED ROOT	RELATIVE ERROR
BISECTION	1.77218	1.77	0.123205
BISECTION	2.50656	2.5	0.261802
BISECTION	3.06998	3.07	0.000584634
BISECTION	3.54491	3.54	0.138486
BISECTION	3.96333	3.96	0.0839357
NEWTON RAPHSON	1.77218	2.5	41.0689
NEWTON RAPHSON	9.19277e-13	1.7	1.84928e+14
NEWTON RAPHSON	3.06998	3.07	0.00059629
NEWTON RAPHSON	3.54491	3.54	0.138484
NEWTON RAPHSON	3.54491	3.96	11.7095
SECANT	1.77218	2.5	41.0689
SECANT	1.41339e-12	1.7	1.20278e+14
SECANT	3.06998	3.07	0.00059629
SECANT	3.06998	3.54	15.3101
SECANT	3.54491	3.96	11.7095

I observed that my predictions were accurate. the Bisection Method is reliable but slow, while Nepton-Raphson is faster and more efficient. The Bisection Method gave precise results are acceptable, though the Fixed-Point Method showed unacceptable errors for the second equation due to its complexity.

In [ ]:

In [ ]:

In [ ]: